Market_Sentiment_Predictor_Pipeline

February 13, 2026

1 Market Sentiment Predictor

Geovanni Jones, Josh Previte, Casey Barrasso

The goal of this project is to predict the directional price movement of a specific stock using both historical price data and news information. Two separate models will predict price movements based on historical data and analyze news sentiment.

[2]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import os
import re
import warnings
from pandas.errors import SettingWithCopyWarning

from collections import Counter

from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.seasonal import STL
```

[3]:
```
os.getcwd()
```

[3]:
```
'/home/jupyter-jonesaop/market-sentiment-predictor/notebooks'
```

[4]:
```
#suppress pesky filter warnings
warnings.filterwarnings("ignore", category=SettingWithCopyWarning)
```

2 Week 2 - Data Ingestion

There are two main datasets to this project. The first is a dataset containing news article content including titles, content, and sentiment scores from various news sources around the AAPL stock. The second dataset (introduced later in the notebook) contains a time series of AAPL stock prices. The final dataset will combine the two datasets on date.

2.1 Sentiment Dataset

[5]:
```
#Github didn't allow us to upload a file more than 25MB. This is combining all␣
 ↪files into one file
raw_data_1 = pd.read_csv('../data/apple_news_data_1.csv', encoding='latin-1')
raw_data_2 = pd.read_csv('../data/apple_news_data_2.csv', encoding='latin-1')
raw_data_3 = pd.read_csv('../data/apple_news_data_3.csv', encoding='latin-1')
raw_data_4 = pd.read_csv('../data/apple_news_data_4.csv', encoding='latin-1')
raw_data_5 = pd.read_csv('../data/apple_news_data_5.csv', encoding='latin-1')

raw_data = pd.concat([raw_data_1, raw_data_2, raw_data_3, raw_data_4,␣
 ↪raw_data_5])[['date', 'title', 'content', 'link', 'symbols', 'tags',␣
 ↪'sentiment_polarity', 'sentiment_neg', 'sentiment_neu', 'sentiment_pos']]
```

[6]:
```
#view a small subsection of the data to ensure formatting looks correct
raw_data.head(10)
```

[6]:
```
                       date  \
0  2021-04-30T21:38:00+00:00
1  2021-04-30T20:41:52+00:00
2  2021-04-30T20:18:00+00:00
3  2021-04-30T19:32:30+00:00
4  2021-04-30T18:49:00+00:00
5  2021-04-30T18:30:06+00:00
6  2021-04-30T18:19:10+00:00
7  2021-04-30T18:13:06+00:00
8  2021-04-30T17:54:31+00:00
9  2021-04-30T17:34:59+00:00

                                               title  \
0  Techâ s Big Five Had Fantastic Pandemics. Her…
1  Valuation Remains a Concern for Churchill Capi…
2  Big techâ s trillion-dollar pandemic year may…
3  What Appleâ s blowout earnings means for Berk…
4  EU Agrees With Spotify: Apple Is Abusing Its A…
5  ETFs to Stack as Reopening Poses No Hindrance …
6  Big Tech Stocks Flash Red Flag to Regulators A…
7  Top Stock Reports for Apple, Alphabet &amp;amp; Bl…
8  Why Stem Stock Looks Very Attractive Following…
9  â Technology is the largest single growth tre…

                                             content  \
0  Alphabet, Amazon.com, Apple, Facebook, and Mic…
1  Churchill Capital IV (NYSE:CCIV) stock has gra…
2  This week concludes the first quarter earnings…
3  Krish Sankar, Managing Dir. & Cowen Senior Res…
4  The EU's European Commission just released pre…
5  The tech-heavyÃ Nasdaq Composite Index was a s…
```

```
6  If â Big Techâ  was a gamefish, like Faceboo…
7  Friday, April 30, 2021\n\nThe Zacks Research D…
8  Not infrequently, Wall Street focuses excessiv…
9  Keith Fitz-Gerald, Fitz-Gerald Group Chief Inv…

                                                 link  \
0  https://finance.yahoo.com/m/bbe3f7c9-c7af-3c9f…
1  https://finance.yahoo.com/news/valuation-remai…
2  https://finance.yahoo.com/m/6e441d74-37c1-3642…
3  https://finance.yahoo.com/video/apple-blowout-…
4  https://finance.yahoo.com/m/07524444-63a7-34aa…
5  https://finance.yahoo.com/news/etfs-stack-reop…
6  https://finance.yahoo.com/news/big-tech-stocks…
7  https://finance.yahoo.com/news/top-stock-repor…
8  https://finance.yahoo.com/news/why-stem-stock-…
9  https://finance.yahoo.com/video/technology-lar…

                                               symbols  \
0       AAPL.US, AAPL34.SA, AMZN.US, FB.US, GOOGL.US
1                        AAPL.US, CCIV-UN.US, CCIV.US
2                AAPL.US, AAPL34.SA, AMZN.US, MSFT.US
3  AAPL.US, AAPL34.SA, BRK-A.US, BRK-B.US, DJI.IN…
4                         AAPL.US, AAPL34.SA, SPOT.US
5  AAPL.US, AAPL34.SA, COMP.US, FB.US, GOOGL.US, …
6  AAPL.US, CRM.US, FB.US, GME.US, GOOGL.US, MSFT…
7  AAPL.US, AAPL34.SA, BLK.US, F.US, GOOGL.US, ZT…
8  AAPL.US, AMZN.US, BX.US, FSLR.US, GE.US, GEOO3…
9  AAPL.US, AAPL34.SA, AMZN.US, DJI.INDX, FB.US, …

                                                 tags sentiment_polarity  \
0  AMAZON, APPLE, FACEBOOK, HARLEY FINKELSTEIN, M…               0.395
1  BUSINESS COMBINATION, CCIV, CHURCHILL CAPITAL …               0.998
2  AMAZON, APPLE INC, BIG TECH, MARKET CAPITALIZA…                   0
3  ALEXIS CHRISTOFOROUS, BERKSHIRE HATHAWAY, COWE…                   0
4  APP STORE, APPLE, EUROPEAN COMMISSION, MUSIC S…               -0.71
5  APPLE, CONSENSUS ESTIMATE, EARNINGS PER SHARE,…               0.999
6  BITCOIN, DIGITAL TRANSFORMATION, FACEBOOK, MIC…               0.999
7  ALPHABET, APPLE, APPLE WATCH, BLACKROCK, RESEA…               0.999
8                          CLEAN-ENERGY, STEM STOCK                 0.999
9  ALEXIS CHRISTOFOROUS, KEITH FITZ-GERALD, SIBIL…                   0

  sentiment_neg sentiment_neu sentiment_pos
0             0         0.871         0.129
1         0.023          0.85         0.127
2             0             1             0
3             0             1             0
4         0.093         0.885         0.022
```

```
5         0.027         0.826         0.148
6         0.026         0.861         0.113
7         0.042         0.775         0.183
8         0.015          0.83         0.154
9             0             1             0
```

[7]:
```
# Sort raw_data by date column from earliest to latest
raw_data = raw_data.sort_values('date', ascending=True).reset_index(drop=True)

#raw_data.head(10)
raw_data.tail(10)
```

[7]:
```
       date title content link symbols tags sentiment_polarity sentiment_neg  \
35594  NaN   NaN     NaN   NaN     NaN  NaN                 NaN           NaN
35595  NaN   NaN     NaN   NaN     NaN  NaN                 NaN           NaN
35596  NaN   NaN     NaN   NaN     NaN  NaN                 NaN           NaN
35597  NaN   NaN     NaN   NaN     NaN  NaN                 NaN           NaN
35598  NaN   NaN     NaN   NaN     NaN  NaN                 NaN           NaN
35599  NaN   NaN     NaN   NaN     NaN  NaN                 NaN           NaN
35600  NaN   NaN     NaN   NaN     NaN  NaN                 NaN           NaN
35601  NaN   NaN     NaN   NaN     NaN  NaN                 NaN           NaN
35602  NaN   NaN     NaN   NaN     NaN  NaN                 NaN           NaN
35603  NaN   NaN     NaN   NaN     NaN  NaN                 NaN           NaN

       sentiment_neu sentiment_pos
35594            NaN           NaN
35595            NaN           NaN
35596            NaN           NaN
35597            NaN           NaN
35598            NaN           NaN
35599            NaN           NaN
35600            NaN           NaN
35601            NaN           NaN
35602            NaN           NaN
35603            NaN           NaN
```

[8]:
```
#check how many NA values there are. We care most about sentiment, date, and␣
 ↪content columns
raw_data.isna().sum().sort_values(ascending=False)
```

[8]:
```
tags                17790
sentiment_pos        5853
sentiment_neu        5812
sentiment_neg        5767
sentiment_polarity   5721
symbols              5539
link                 5340
```

```
content                 4901
title                   4255
date                    2808
dtype: int64
```

[9]:
```
#Drop Clumns we won't be focusing on (link and tags)
raw_data = raw_data.drop(columns=['link', 'tags'])
```

[10]:
```
# force sentiment scores to be a numerical value, NA if not
for col in␣
 ↪['sentiment_pos','sentiment_neu','sentiment_neg','sentiment_polarity']:
    raw_data[col] = pd.to_numeric(raw_data[col], errors='coerce')

# extract valid calendar dates only (ignore time + BST noise)
raw_data['date'] = pd.to_datetime(
    raw_data['date'].astype(str).str.extract(r'(\d{4}-\d{2}-\d{2})')[0],
    errors='coerce'
) #This section of the code was inputed to handle warnings from pandas that␣
 ↪could lead to future breakage of the code.

# convert content to string
raw_data['content'] = raw_data['content'].astype('str')
```

[11]:
```
# Setting "date" column to datetime format
raw_data['date'] = pd.to_datetime(raw_data['date'])

print(raw_data.shape)

#Data range for raw_data
print(f"Date range for raw_data: {raw_data['date'].min()} to {raw_data['date'].
 ↪max()}")
```

```
(35604, 8)
Date range for raw_data: 2016-02-19 00:00:00 to 2024-11-27 00:00:00
```

[12]:
```
#this data is messy, drop observations that have NAs in critical columns
raw_data = raw_data.
 ↪dropna(subset=['sentiment_pos','sentiment_neu','sentiment_neg','sentiment_polarity','conten

#print out the data type of each column to ensure conversions from above worked
for col in raw_data.columns:
    print(f'Column: {col} , Data Type: {raw_data[col].dtype}')
```

```
Column: date , Data Type: datetime64[ns]
Column: title , Data Type: object
Column: content , Data Type: object
Column: symbols , Data Type: object
Column: sentiment_polarity , Data Type: float64
```

```
Column: sentiment_neg , Data Type: float64
Column: sentiment_neu , Data Type: float64
Column: sentiment_pos , Data Type: float64
```

[13]:
```
#Create a single sentiment score that we can use to determine if an article␣
 ↪contains more negative or positive sentiment
raw_data['Net Sentiment'] = raw_data['sentiment_pos'] -␣
 ↪raw_data['sentiment_neg']

#while not the main target variable, net sentiment could be interesting to␣
 ↪determine presence of positive and negative language
#we can also make this a classification model by coverting to 'Positive',␣
 ↪'Neutral', or 'Negative' here.
```

The "content" column will be the predictor variable in the first model, and the string will be transformed into a numerical matrix using feature engineering methods such as TF-IDF. The "snetiment polarity" variable will be the target variable. The goal of the first model will be to predict the sentiment polarity of a news article given its content.

[14]:
```
print(raw_data.shape)
```

```
(29700, 9)
```

[15]:
```
# Dropping rows before January 1, 2018 and after October 31, 2024
raw_data = raw_data[(raw_data['date'] >= '2018-01-01') & (raw_data['date'] <=␣
 ↪'2024-10-31')]

print(raw_data.shape)

#Data range for raw_data
print(f"Date range for raw_data: {raw_data['date'].min()} to {raw_data['date'].
 ↪max()}")
```

```
(29635, 9)
Date range for raw_data: 2018-01-31 00:00:00 to 2024-10-31 00:00:00
```

This dataset has 29,638 rows and 10 columns (plus the net sentiment score column created by us)

2.2 Price Timeseries Dataset

[16]:
```
ts_data = pd.read_csv('../data/AAPL Stock Data.csv')
```

[17]:
```
ts_data.head(10)
```

[17]:

	Dates	AAPL Share Price
0	Jan-03-2005	1.13
1	Jan-04-2005	1.14
2	Jan-05-2005	1.15
3	Jan-06-2005	1.15

```
4  Jan-07-2005              1.24
5  Jan-10-2005              1.23
6  Jan-11-2005              1.15
7  Jan-12-2005              1.17
8  Jan-13-2005              1.25
9  Jan-14-2005              1.25
```

[18]:
```
#check how many NA values there are
ts_data.isna().sum().sort_values(ascending=False)
```

[18]:
```
Dates              0
AAPL Share Price   0
dtype: int64
```

[19]:
```
#Convert values to the expected formatting
ts_data['Dates'] = pd.to_datetime(ts_data['Dates']).dt.date
ts_data['AAPL Share Price'] = ts_data['AAPL Share Price'].astype('float')
```

[20]:
```
for col in ts_data.columns:
    print(f'Column: {col} , Data Type: {type(ts_data[col][0])}')
```

```
Column: Dates , Data Type: <class 'datetime.date'>
Column: AAPL Share Price , Data Type: <class 'numpy.float64'>
```

[21]:
```
print(ts_data.shape)
```

```
(5298, 2)
```

[22]:
```
# Setting "Dates" column to datetime format
ts_data['Dates'] = pd.to_datetime(ts_data['Dates'])

# Dropping rows before January 1, 2018 and after October 31, 2024
ts_data = ts_data[(ts_data['Dates'] >= '2018-01-01') & (ts_data['Dates'] <=␣
 ↪'2024-10-31')]

print(ts_data.shape)
print(f"Date range: {ts_data['Dates'].min()} to {ts_data['Dates'].max()}")
```

```
(1720, 2)
Date range: 2018-01-02 00:00:00 to 2024-10-31 00:00:00
```

There are 1,720 rows in this dataset and two columns (date and price).

2.3 Combining Datasets

[23]:
```
# Recreate sentiment_data from raw_data to ensure clean state
sentiment_data = raw_data.groupby(['date'])[['sentiment_polarity']].mean()

# Create a complete date range from January 1, 2018 to October 31, 2024
```

```
date_range = pd.date_range(start='2018-01-01', end='2024-10-31', freq='D')
complete_dates = pd.DataFrame({'date': date_range})

# Reset index to make 'date' a column
sentiment_data = sentiment_data.reset_index()

# Convert to datetime
sentiment_data['date'] = pd.to_datetime(sentiment_data['date'])

# Merge with complete date range
sentiment_data = pd.merge(complete_dates, sentiment_data, on='date', how='left')

#Set index back to date column
sentiment_data = sentiment_data.set_index('date')

print(f"Sentiment data shape after filling missing dates: {sentiment_data.
 ↪shape}")
sentiment_data.head(10)
```

```
Sentiment data shape after filling missing dates: (2496, 1)
```

[23]:

date	sentiment_polarity
2018-01-01	NaN
2018-01-02	NaN
2018-01-03	NaN
2018-01-04	NaN
2018-01-05	NaN
2018-01-06	NaN
2018-01-07	NaN
2018-01-08	NaN
2018-01-09	NaN
2018-01-10	NaN

[24]:
```
#join on date
all_data = pd.merge(sentiment_data, ts_data, how = 'right', left_on = 'date',␣
 ↪right_on = 'Dates')
```

[25]:
```
for col in all_data.columns:
    print(f'Column: {col} , Data Type: {type(all_data[col][0])}')
```

```
Column: sentiment_polarity , Data Type: <class 'numpy.float64'>
Column: Dates , Data Type: <class 'pandas._libs.tslibs.timestamps.Timestamp'>
Column: AAPL Share Price , Data Type: <class 'numpy.float64'>
```

[26]:
```
all_data.head(10)
```

[26]:

	sentiment_polarity	Dates	AAPL Share Price
0	NaN	2018-01-02	43.07
1	NaN	2018-01-03	43.06
2	NaN	2018-01-04	43.26
3	NaN	2018-01-05	43.75
4	NaN	2018-01-08	43.59
5	NaN	2018-01-09	43.58
6	NaN	2018-01-10	43.57
7	NaN	2018-01-11	43.82
8	NaN	2018-01-12	44.27
9	NaN	2018-01-16	44.05

We have a greater amount of stock price data than news sentiment data on the AAPL stock. There are currently 5,298 rows and 3 columns, but only 1,574 rows have sentiment data (as shown from the print statement above). We can remove dates without any sentiment analysis at a future date. For right now, this data could be helpful for a traditional forecasting model

[27]:
```
all_data['sentiment_polarity'].describe()
```

[27]:
```
count    1112.000000
mean        0.531748
std         0.299017
min        -0.993000
25%         0.412746
50%         0.554172
75%         0.685809
max         1.000000
Name: sentiment_polarity, dtype: float64
```

The sentiment polarity looks to be performing as expected. Scores range from -.99 to 1.00, but most are positive (which is expected due to Apple's stellar performance during this time period.

All models will use next-day stock price direction as the main target variable. There will be three possible forecasting models, the traditional model will exclusively use previous stock prices as predictor variables, the second model exclusively uses sentiment analysis as the predictor variable, while the third model will combine both.

[28]:
```
#Data range for all_data
print(f"Date range for all_data: {all_data['Dates'].min()} to␣
 ↪{all_data['Dates'].max()}")
```

```
Date range for all_data: 2018-01-02 00:00:00 to 2024-10-31 00:00:00
```

3 Week 3 - Split and EDA

This week, we will focus on splitting the data into a train, validation, and test set and performing exploratory data analysis (EDA).

3.1 Split Sentiment Dataset

[29]:
```
raw_data = raw_data.sort_values("date").reset_index(drop=True)

raw_train = raw_data[(raw_data["date"] >= "2018-01-01") & (raw_data["date"] <=␣
 ↪"2022-10-31")]
raw_val   = raw_data[(raw_data["date"] >= "2022-11-01") & (raw_data["date"] <=␣
 ↪"2023-10-31")]
raw_test  = raw_data[(raw_data["date"] >= "2023-11-01") & (raw_data["date"] <=␣
 ↪"2024-10-31")]

print(raw_train.shape)
print(raw_val.shape)
print(raw_test.shape)

raw_train.head(10)
```

```
(15474, 9)
(7598, 9)
(6563, 9)
```

[29]:

	date	title	content	symbols	sentiment_polarity
0	2018-01-31	Investor Expectations to Drive Momentum within…	NEW YORK, Jan. 31, 2018 (GLOBE NEWSWIRE) -- …	AAPL.US, AVHI.US, FARM.US, GM.US, SGRY.US	0.995
1	2018-03-16	Top 100 Reputable Companies Around the Globe A…	Boston, MA, March 16, 2018 (GLOBE NEWSWIRE) …	AAPL.US, AMZN.US, BA.US, BUD.US, CAJ.US, CAT.U…	0.991
2	2018-03-27	Universal Display Corporation Stock Is Way Und…	Since topping out in January, shares of Univer…		
3	2018-04-16	Detailed Research: Economic Perspectives on Ge…	NEW YORK, April 16, 2018 (GLOBE NEWSWIRE) --…		
4	2018-06-22	Apple iPhone Spared Tariffs, But Could Face Ch…	In an escalating trade war between China and t…		
5	2018-10-23	Report: Exploring Fundamental Drivers Behind T…	NEW YORK, Oct. 23, 2018 (GLOBE NEWSWIRE) -- …		
6	2018-11-30	New Research Coverage Highlights Omeros, Apple…	NEW YORK, Nov. 30, 2018 (GLOBE NEWSWIRE) -- …		
7	2019-01-03	Apple, AAPL Investment Losses Alert: Bernstein…	NEW YORK, Jan. 02, 2019 (GLOBE NEWSWIRE) -- …		
8	2019-01-03	SHAREHOLDER ALERT: Bronstein, Gewirtz & Grossm…	NEW YORK, Jan. 03, 2019 (GLOBE NEWSWIRE) -- …		
9	2019-02-22	Factors of Influence in 2019, Key Indicators a…	NEW YORK, Feb. 22, 2019 (GLOBE NEWSWIRE) -- …		

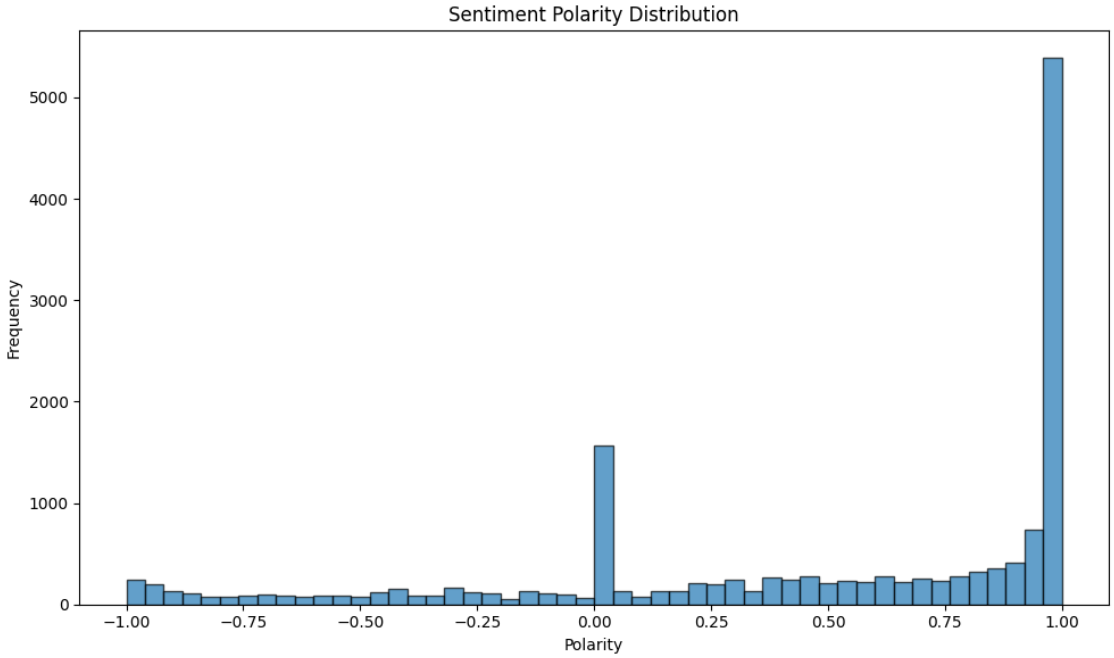
```
2                                    AAPL.US, OLED.US              -0.244
3      AAPL.US, FB.US, GD.US, GRMN.US, HD.US, WBA.US               0.997
4                                  AAPL.US, HNHPF.US              -0.807
5  AAPL.US, BTX.US, CTLT.US, HD.US, PDLI.US, QURE.US               0.995
6    AAPL.US, BIO.US, COT.US, DAR.US, OMER.US, OR.US               0.998
7                                             AAPL.US              0.992
8                                             AAPL.US              0.967
9  AAPL.US, CBAY.US, FTK.US, GVA.US, IMGN.US, UAL.US               0.997

   sentiment_neg  sentiment_neu  sentiment_pos  Net Sentiment
0          0.009          0.937          0.054          0.045
1          0.011          0.917          0.072          0.061
2          0.134          0.764          0.101         -0.033
3          0.008          0.930          0.062          0.054
4          0.138          0.862          0.000         -0.138
5          0.008          0.941          0.051          0.043
6          0.009          0.925          0.066          0.057
7          0.038          0.816          0.147          0.109
8          0.050          0.839          0.111          0.061
9          0.009          0.928          0.064          0.055
```

3.2 EDA Sentiment Dataset

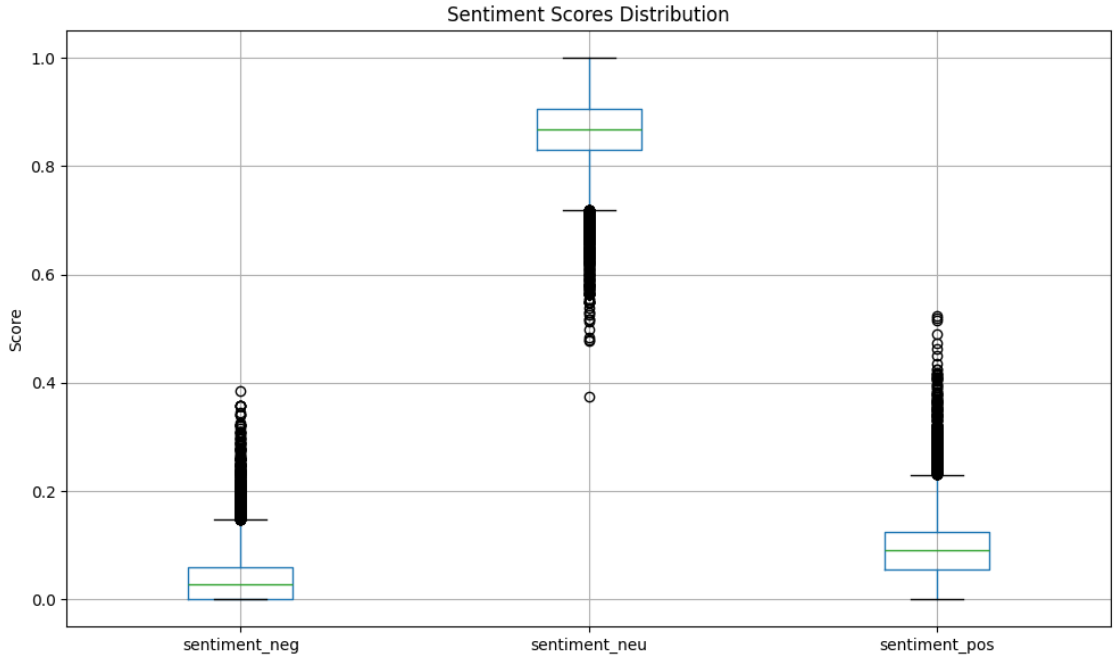
Visual Analysis:

- Sentiment polarity distribution histogram
- Sentiment scores comparison (neg/neu/pos) boxplot
- Articles count over time (monthly trend)
- Average sentiment polarity over time with zero baseline

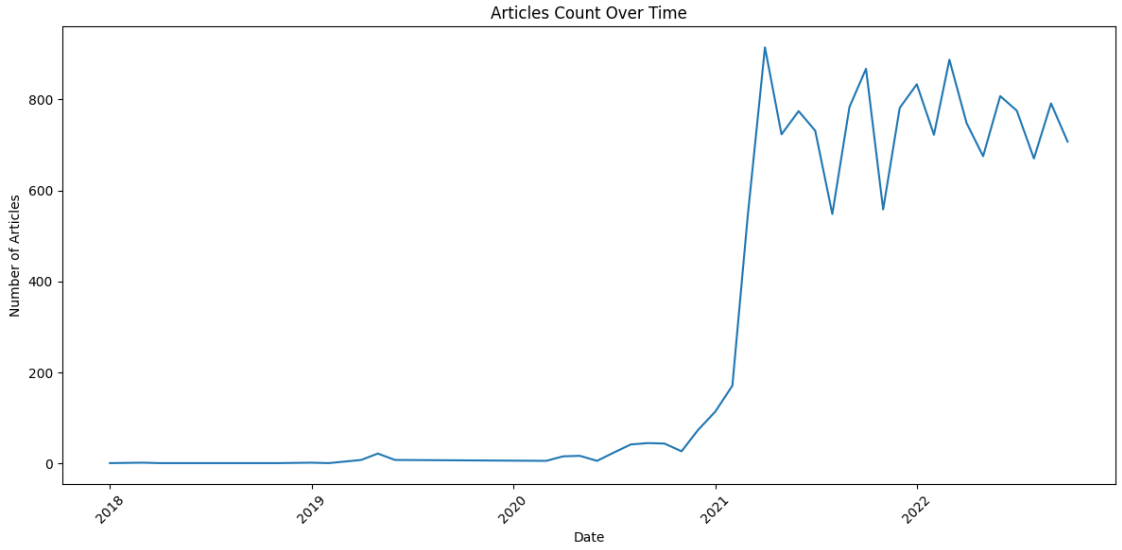
[30]:
```python
# Sentiment polarity distribution
plt.figure(figsize=(10, 6))
plt.hist(raw_train['sentiment_polarity'], bins=50, edgecolor='black', alpha=0.7)
plt.title('Sentiment Polarity Distribution')
plt.xlabel('Polarity')
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()
```



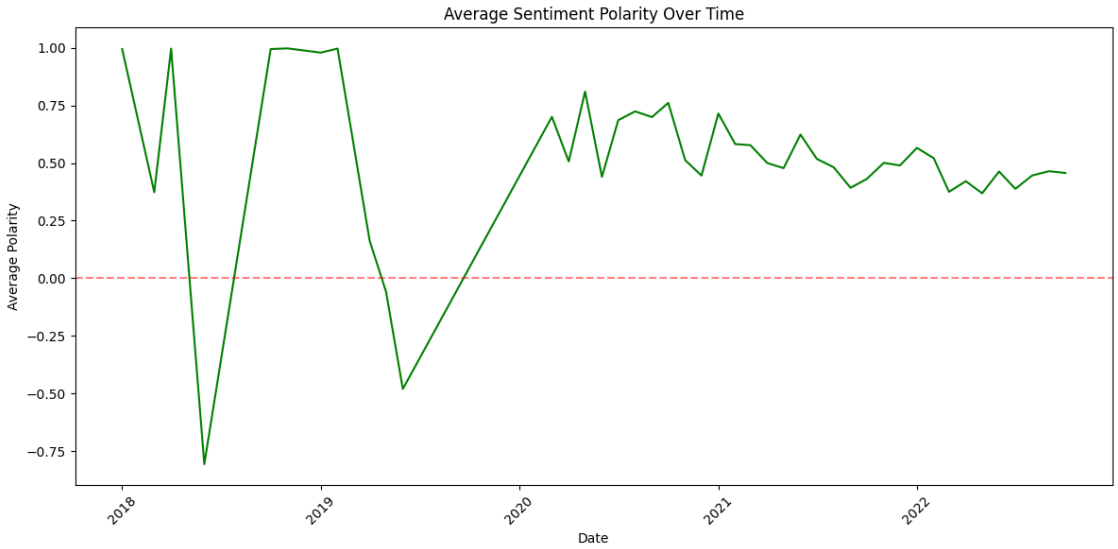
[31]:
```
# Sentiment scores comparison
plt.figure(figsize=(10, 6))
sentiment_cols = ['sentiment_neg', 'sentiment_neu', 'sentiment_pos']
raw_train[sentiment_cols].boxplot()
plt.title('Sentiment Scores Distribution')
plt.ylabel('Score')
plt.tight_layout()
plt.show()
```



[32]:
```
# Articles per date
raw_train['date'] = pd.to_datetime(raw_train['date'])
articles_per_date = raw_train.groupby(raw_train['date'].dt.to_period('M')).
 ↪size()
plt.figure(figsize=(12, 6))
plt.plot(articles_per_date.index.to_timestamp(), articles_per_date.values)
plt.title('Articles Count Over Time')
plt.xlabel('Date')
plt.ylabel('Number of Articles')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



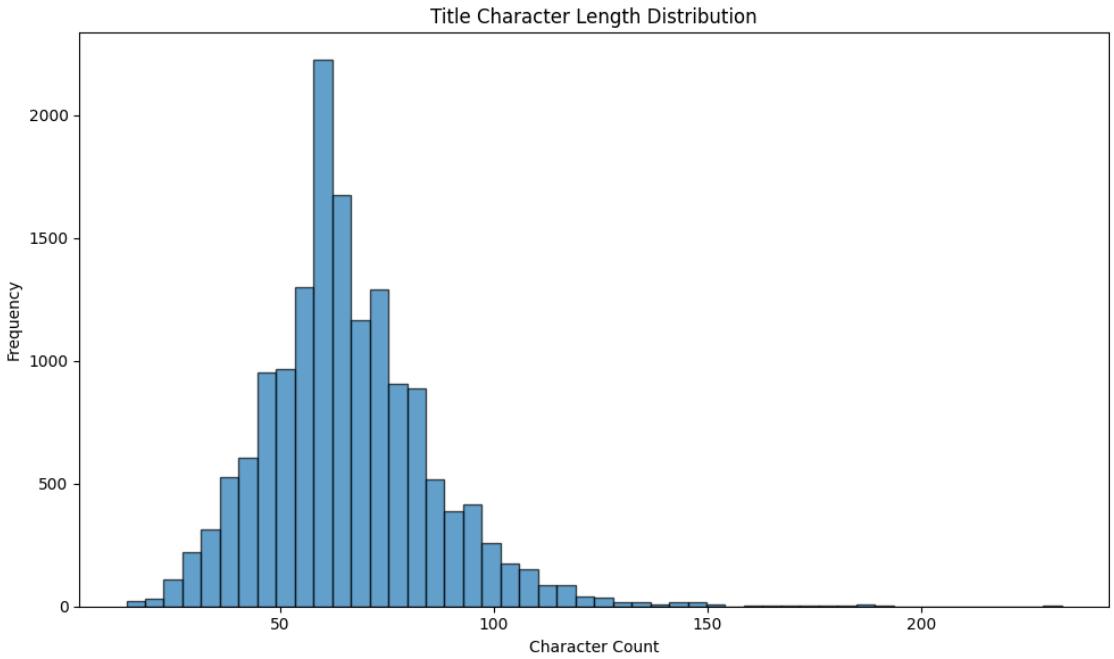
[33]:
```
# Average sentiment over time
avg_sentiment = raw_train.groupby(raw_train['date'].dt.
 ↪to_period('M'))['sentiment_polarity'].mean()
plt.figure(figsize=(12, 6))
plt.plot(avg_sentiment.index.to_timestamp(), avg_sentiment.values,␣
 ↪color='green')
plt.title('Average Sentiment Polarity Over Time')
plt.xlabel('Date')
plt.ylabel('Average Polarity')
plt.axhline(y=0, color='r', linestyle='--', alpha=0.5)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



NLP Analysis on Title:

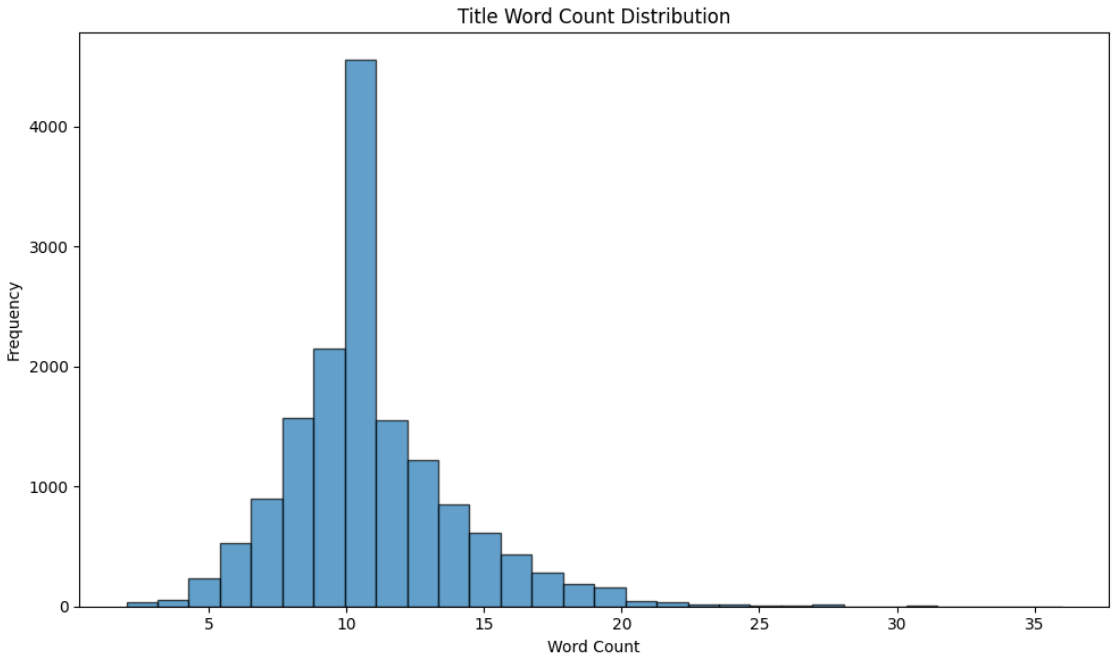
- Character length and word count distributions
- Average statistics

[34]:
```
# Title length distribution
raw_train['title_length'] = raw_train['title'].fillna('').str.len()
    # Title word count

# Title character length
plt.figure(figsize=(10, 6))
plt.hist(raw_train['title_length'], bins=50, edgecolor='black', alpha=0.7)
plt.title('Title Character Length Distribution')
plt.xlabel('Character Count')
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()

print(f"\nAverage title length: {raw_train['title_length'].mean():.2f}
characters")
```




```
Average title length: 65.83 characters
```

[35]:
```
# Title word count
raw_train['title_word_count'] = raw_train['title'].fillna('').str.split().str.
 ↪len()

# Title word count
plt.figure(figsize=(10, 6))
plt.hist(raw_train['title_word_count'], bins=30, edgecolor='black', alpha=0.7)
plt.title('Title Word Count Distribution')
plt.xlabel('Word Count')
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()

print(f"Average title word count: {raw_train['title_word_count'].mean():.2f}␣
 ↪words")
```




```
Average title word count: 10.85 words
```

- Top 20 most common words
- NLP Analysis on Content:

Character length and word count distributions Average statistics and missing content count Correlation Analysis:

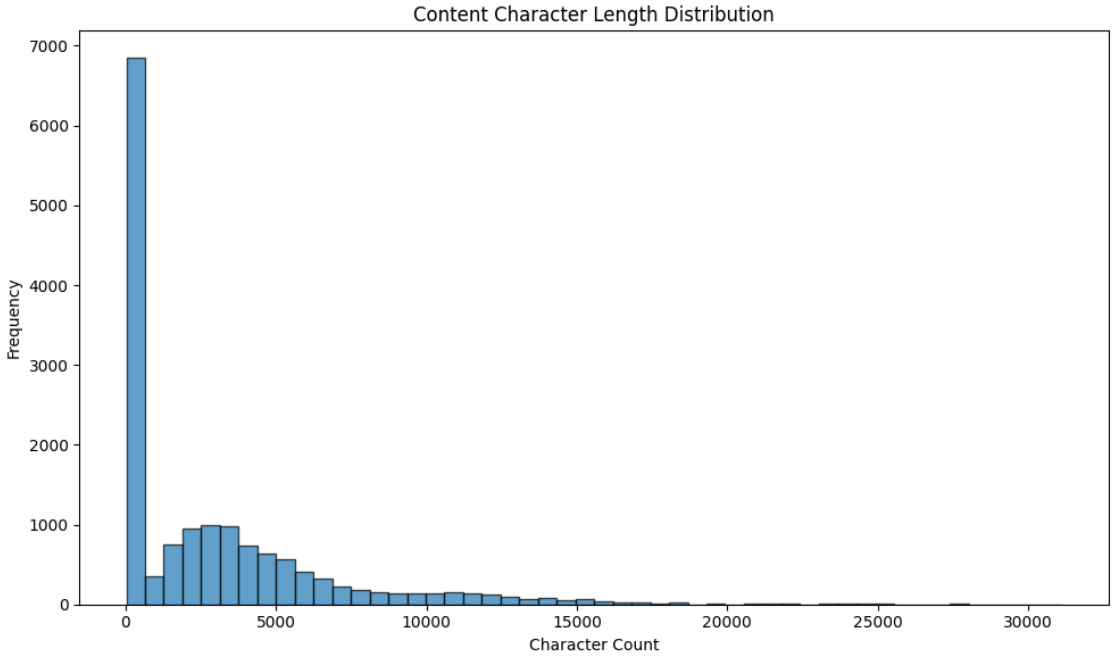
Sentiment vs title/content length scatter plots

[36]:
```
# Most common words in titles
all_title_words = ' '.join(
    raw_train['title']
    .fillna('')
    .astype(str)   # AI Added code to help prevent code breakage in future due␣
 ↪to warning errors.
).lower()

words = re.findall(r'\b[a-z]+\b', all_title_words)
word_freq = Counter(words)
common_words = word_freq.most_common(20)

print("\nTop 20 most common words in titles:")
for word, count in common_words:
    print(f"  {word}: {count}")
```

```
Top 20 most common words in titles:
```

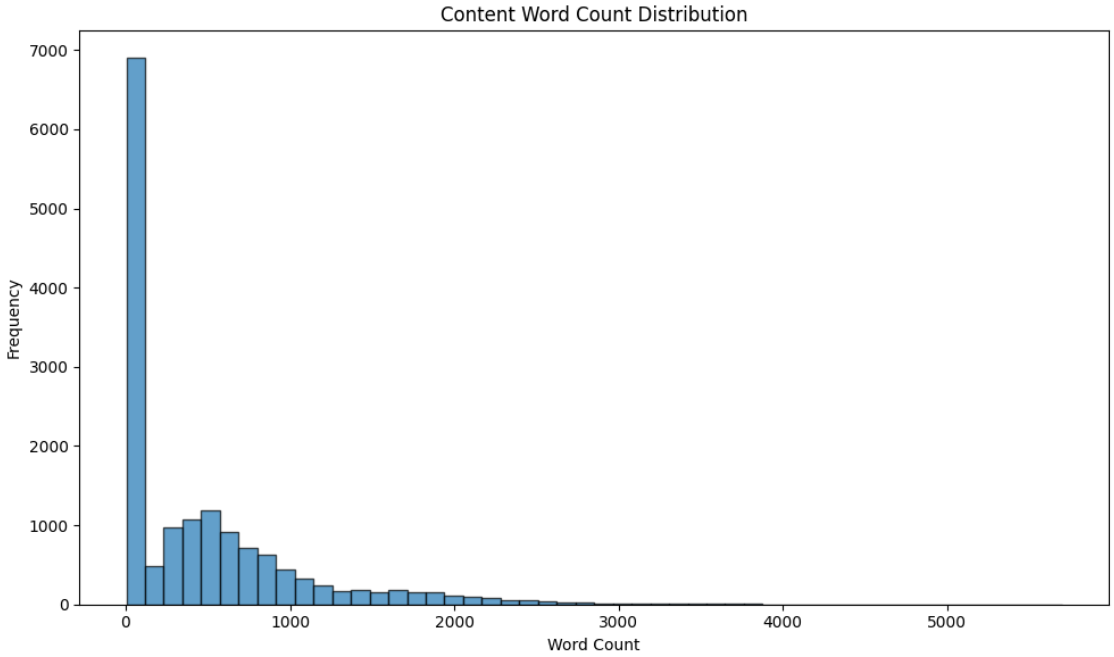
```
apple: 5885
to: 5033
s: 3821
stocks: 2929
the: 2874
in: 2615
stock: 2316
and: 1933
on: 1873
as: 1798
for: 1730
buy: 1617
of: 1600
a: 1587
is: 1494
tech: 1490
market: 1359
dow: 1321
earnings: 1098
with: 973
```

[37]:
```python
# Content length distribution
raw_train.loc[:, 'content_length'] = (
    raw_train['content']
    .fillna('')
    .astype(str)
    .str.len()
) # AI Added code to help prevent code breakage in future due to warning errors.

# Content character length
plt.figure(figsize=(10, 6))
plt.hist(raw_train['content_length'], bins=50, edgecolor='black', alpha=0.7)
plt.title('Content Character Length Distribution')
plt.xlabel('Character Count')
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()

print(f"\nAverage content length: {raw_train['content_length'].mean():.2f}␣
 ↪characters")
```



Average content length: 3020.90 characters

[38]:
```
# Content word count
raw_train.loc[:, 'content_word_count'] = (
    raw_train['content']
    .fillna('')
    .astype(str)
    .str.split()
    .str.len()
) # AI Added code to help prevent code breakage in future due to warning errors.

# Content word count distribution
plt.figure(figsize=(10, 6))
plt.hist(raw_train['content_word_count'], bins=50, edgecolor='black', alpha=0.7)
plt.title('Content Word Count Distribution')
plt.xlabel('Word Count')
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()

print(f"Average content word count: {raw_train['content_word_count'].mean():.
  ↪2f} words")
```

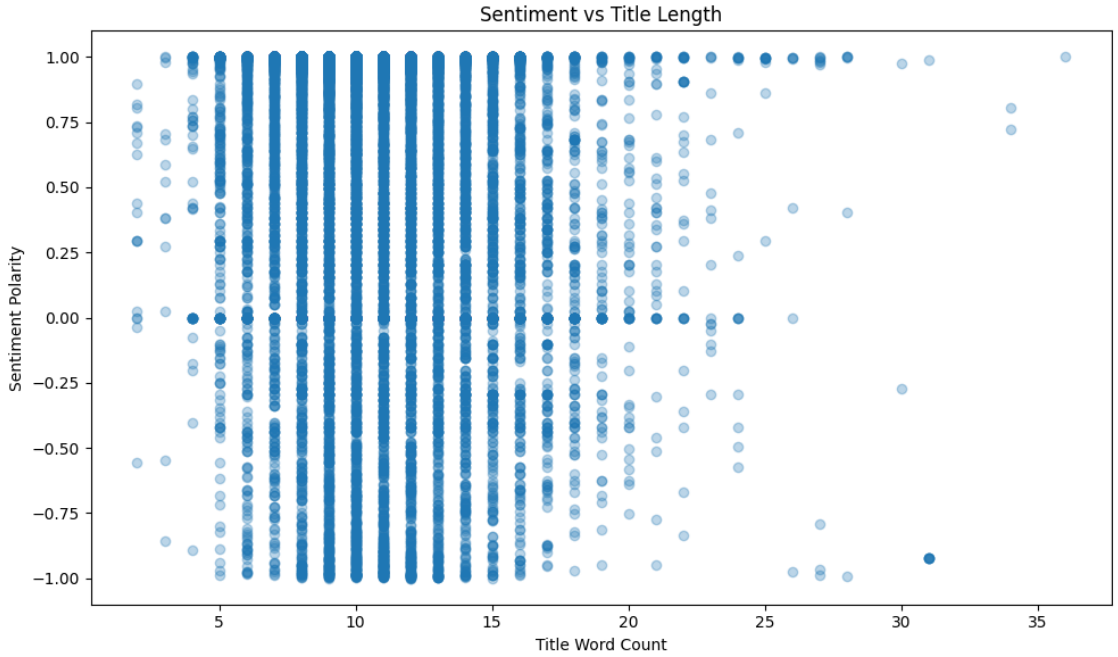



```
Average content word count: 481.55 words
```

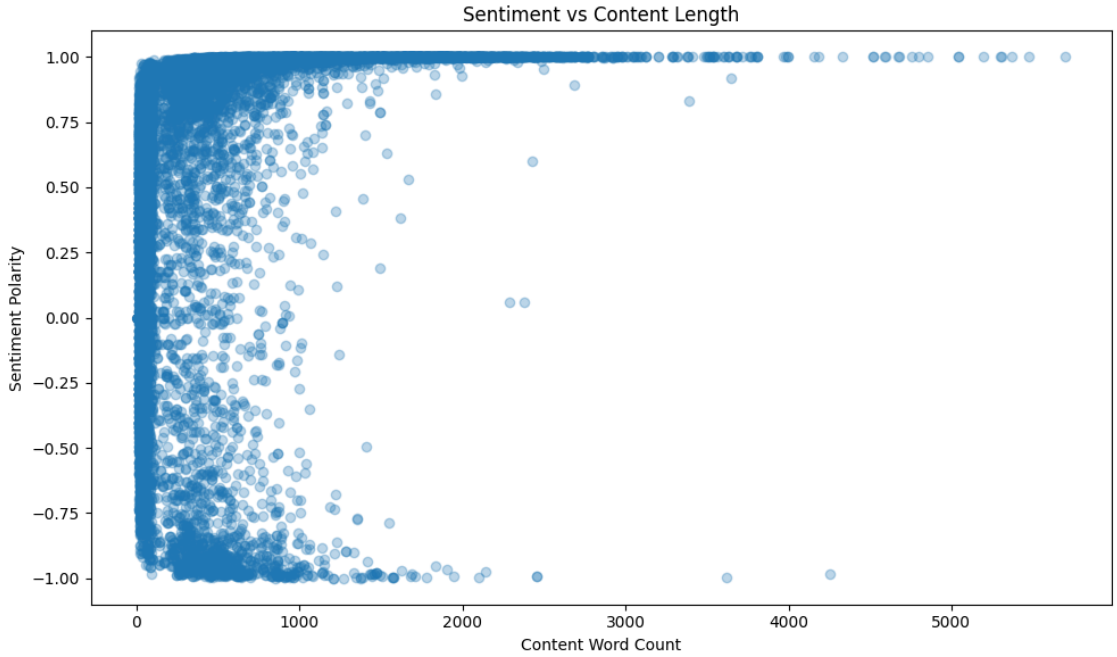
[39]:
```
print(f"Missing content: {raw_train['content'].isna().sum()} articles")
```

```
Missing content: 0 articles
```

[40]:
```
# Sentiment vs Title Length
plt.figure(figsize=(10, 6))
plt.scatter(raw_train['title_word_count'], raw_train['sentiment_polarity'],␣
 ↪alpha=0.3)
plt.title('Sentiment vs Title Length')
plt.xlabel('Title Word Count')
plt.ylabel('Sentiment Polarity')
plt.tight_layout()
plt.show()
```



[41]:
```
# Sentiment vs Content Length
plt.figure(figsize=(10, 6))
plt.scatter(raw_train['content_word_count'], raw_train['sentiment_polarity'],␣
↪alpha=0.3)
plt.title('Sentiment vs Content Length')
plt.xlabel('Content Word Count')
plt.ylabel('Sentiment Polarity')
plt.tight_layout()
plt.show()
```



3.3 Split Price Dataset

[42]:
```
ts_data = ts_data.sort_values("Dates").reset_index(drop=True)

ts_train = ts_data[(ts_data["Dates"] >= "2018-01-01") & (ts_data["Dates"] <=␣
 ↪"2022-10-31")]
ts_val   = ts_data[(ts_data["Dates"] >= "2022-11-01") & (ts_data["Dates"] <=␣
 ↪"2023-10-31")]
ts_test  = ts_data[(ts_data["Dates"] >= "2023-11-01") & (ts_data["Dates"] <=␣
 ↪"2024-10-31")]

print(ts_train.shape)
print(ts_val.shape)
print(ts_test.shape)

ts_train.head(10)
```

```
(1217, 2)
(251, 2)
(252, 2)
```

[42]:

	Dates	AAPL Share Price
0	2018-01-02	43.07
1	2018-01-03	43.06
2	2018-01-04	43.26

```
3 2018-01-05            43.75
4 2018-01-08            43.59
5 2018-01-09            43.58
6 2018-01-10            43.57
7 2018-01-11            43.82
8 2018-01-12            44.27
9 2018-01-16            44.05
```

3.4 EDA Price Dataset

[43]:
```
ts_train.describe()
```

[43]:
```
                                Dates  AAPL Share Price
count                            1217       1217.000000
mean   2020-06-01 17:34:15.875102720         96.583131
min              2018-01-02 00:00:00         35.550000
25%              2019-03-20 00:00:00         50.690000
50%              2020-06-03 00:00:00         81.300000
75%              2021-08-17 00:00:00        142.000000
max              2022-10-31 00:00:00        182.010000
std                              NaN         46.350782
```

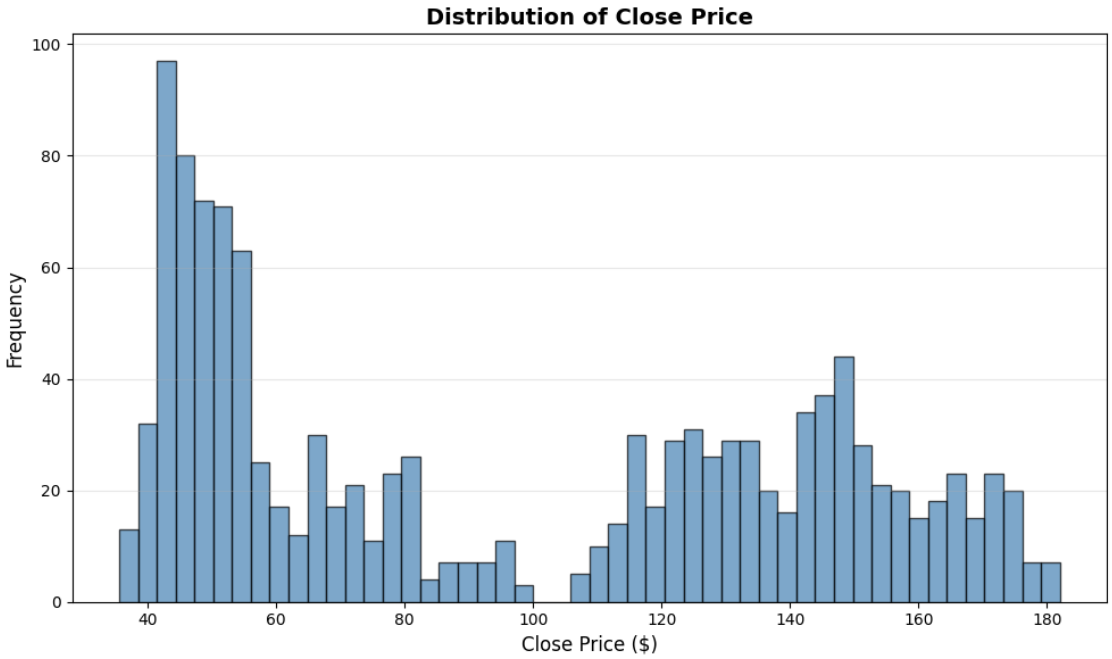
[44]:
```
# Check for missing values
print("Missing values in ts_train:")
print(ts_train.isnull().sum())
print(f"\nPercentage of missing values:")
print((ts_train.isnull().sum() / len(ts_train) * 100).round(2))
```

```
Missing values in ts_train:
Dates               0
AAPL Share Price    0
dtype: int64

Percentage of missing values:
Dates               0.0
AAPL Share Price    0.0
dtype: float64
```

[45]:
```
# Time Series Plot of Close Price
plt.figure(figsize=(15, 6))
plt.plot(ts_train['Dates'], ts_train['AAPL Share Price'], linewidth=1.5,␣
 ↪color='blue')
plt.title('Stock Close Price Over Time (Training Data)', fontsize=16,␣
 ↪fontweight='bold')
plt.xlabel('Date', fontsize=12)
plt.ylabel('Close Price ($)', fontsize=12)
plt.grid(True, alpha=0.3)
plt.tight_layout()
```

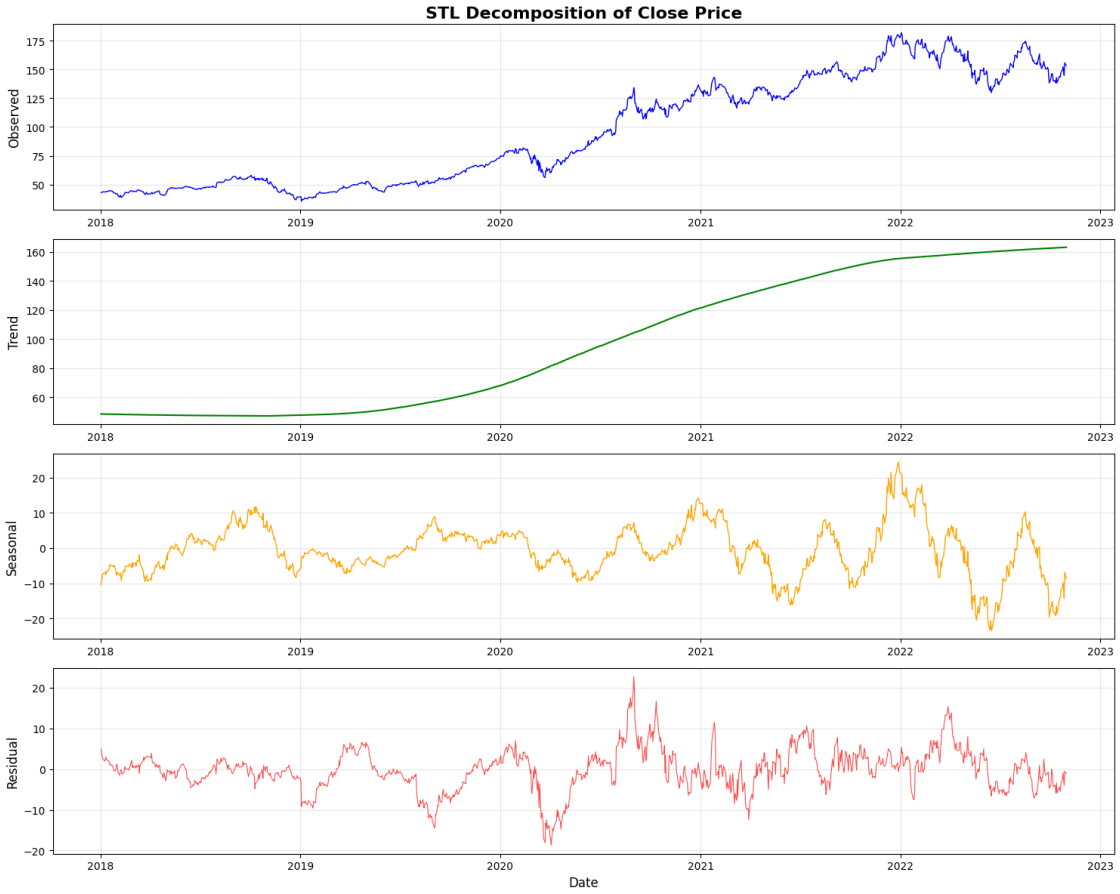
```
plt.show()
```

[46]:
```
# Distribution of Close Price
plt.figure(figsize=(10, 6))

# Histogram
plt.hist(ts_train['AAPL Share Price'], bins=50, color='steelblue',␣
 ↪edgecolor='black', alpha=0.7)
plt.title('Distribution of Close Price', fontsize=14, fontweight='bold')
plt.xlabel('Close Price ($)', fontsize=12)
plt.ylabel('Frequency', fontsize=12)
plt.grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()
```



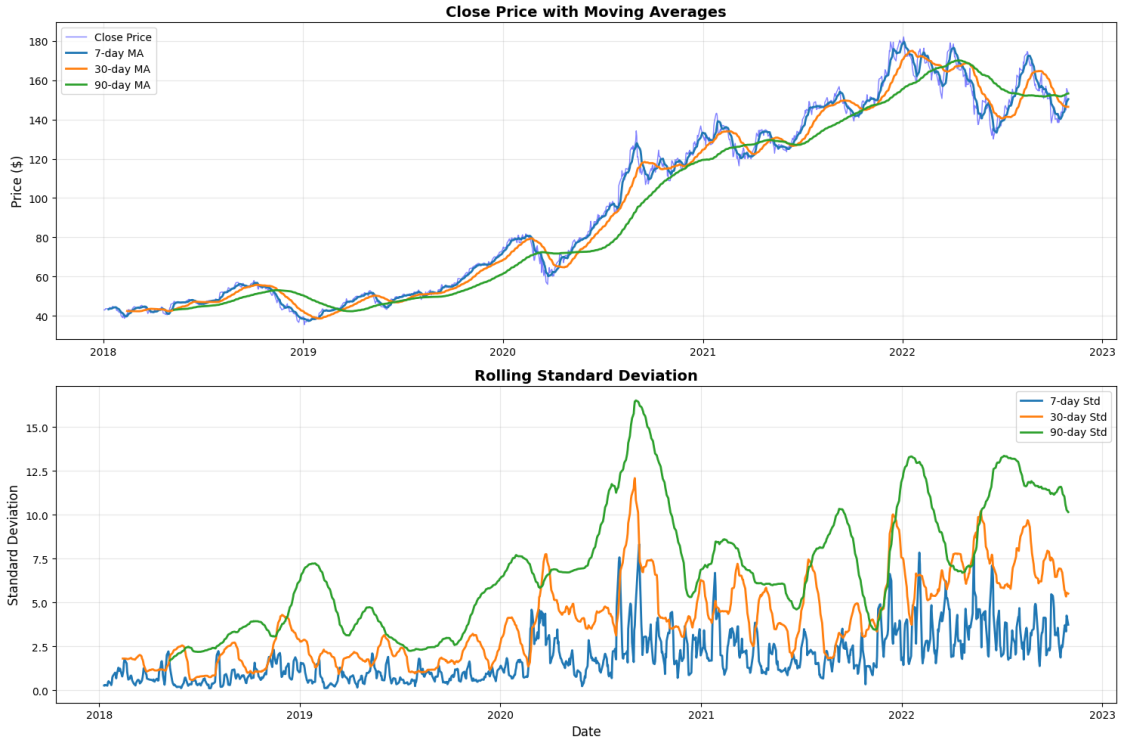
[47]:
```
# STL Decomposition (Seasonal-Trend decomposition using LOESS)
# Prepare time series for STL
ts_for_stl = ts_train.set_index('Dates')['AAPL Share Price']

# Perform STL decomposition
stl = STL(ts_for_stl, seasonal=13, period=252)  # 252 trading days per year
stl_result = stl.fit()

# Plot decomposition
fig, axes = plt.subplots(4, 1, figsize=(15, 12))

# Original
axes[0].plot(stl_result.observed, color='blue', linewidth=1)
axes[0].set_ylabel('Observed', fontsize=12)
axes[0].set_title('STL Decomposition of Close Price', fontsize=16,␣
 ↪fontweight='bold')
axes[0].grid(True, alpha=0.3)

# Trend
axes[1].plot(stl_result.trend, color='green', linewidth=1.5)
axes[1].set_ylabel('Trend', fontsize=12)
axes[1].grid(True, alpha=0.3)

# Seasonal
axes[2].plot(stl_result.seasonal, color='orange', linewidth=1)
```

```
axes[2].set_ylabel('Seasonal', fontsize=12)
axes[2].grid(True, alpha=0.3)

# Residual
axes[3].plot(stl_result.resid, color='red', linewidth=0.8, alpha=0.7)
axes[3].set_ylabel('Residual', fontsize=12)
axes[3].set_xlabel('Date', fontsize=12)
axes[3].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

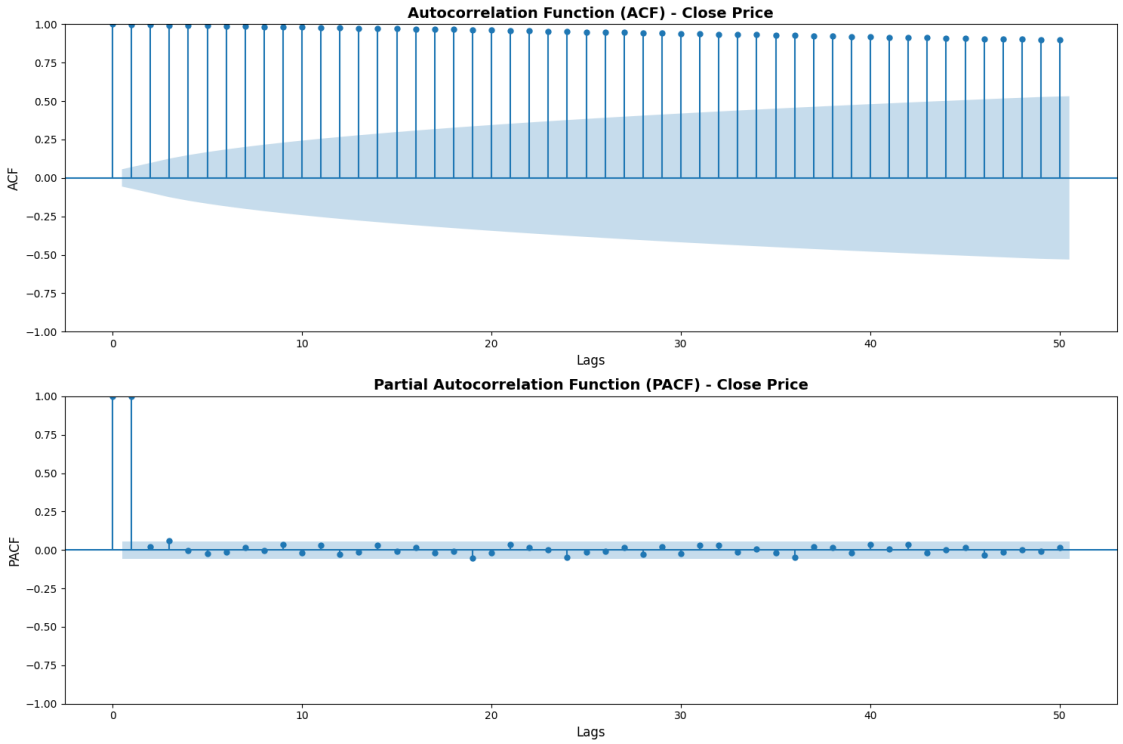


[48]:
```
# Rolling Statistics (Moving Average and Standard Deviation)
window_sizes = [7, 30, 90]  # 1 week, 1 month, 3 months

fig, axes = plt.subplots(2, 1, figsize=(15, 10))

# Rolling Mean
```

```
axes[0].plot(ts_train['Dates'], ts_train['AAPL Share Price'], label='Close␣
 ↪Price', linewidth=1, alpha=0.5, color='blue')
for window in window_sizes:
    rolling_mean = ts_train['AAPL Share Price'].rolling(window=window).mean()
    axes[0].plot(ts_train['Dates'], rolling_mean, label=f'{window}-day MA',␣
 ↪linewidth=2)
axes[0].set_title('Close Price with Moving Averages', fontsize=14,␣
 ↪fontweight='bold')
axes[0].set_ylabel('Price ($)', fontsize=12)
axes[0].legend(loc='best')
axes[0].grid(True, alpha=0.3)

# Rolling Standard Deviation
for window in window_sizes:
    rolling_std = ts_train['AAPL Share Price'].rolling(window=window).std()
    axes[1].plot(ts_train['Dates'], rolling_std, label=f'{window}-day Std',␣
 ↪linewidth=2)
axes[1].set_title('Rolling Standard Deviation', fontsize=14, fontweight='bold')
axes[1].set_xlabel('Date', fontsize=12)
axes[1].set_ylabel('Standard Deviation', fontsize=12)
axes[1].legend(loc='best')
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



[49]:
```
# ACF and PACF Plots
fig, axes = plt.subplots(2, 1, figsize=(15, 10))

# ACF
plot_acf(ts_train['AAPL Share Price'].dropna(), lags=50, ax=axes[0])
axes[0].set_title('Autocorrelation Function (ACF) - Close Price', fontsize=14,␣
 ↪fontweight='bold')
axes[0].set_xlabel('Lags', fontsize=12)
axes[0].set_ylabel('ACF', fontsize=12)

# PACF
plot_pacf(ts_train['AAPL Share Price'].dropna(), lags=50, ax=axes[1])
axes[1].set_title('Partial Autocorrelation Function (PACF) - Close Price',␣
 ↪fontsize=14, fontweight='bold')
axes[1].set_xlabel('Lags', fontsize=12)
axes[1].set_ylabel('PACF', fontsize=12)

plt.tight_layout()
plt.show()
```



3.5 Split Combined Dataset

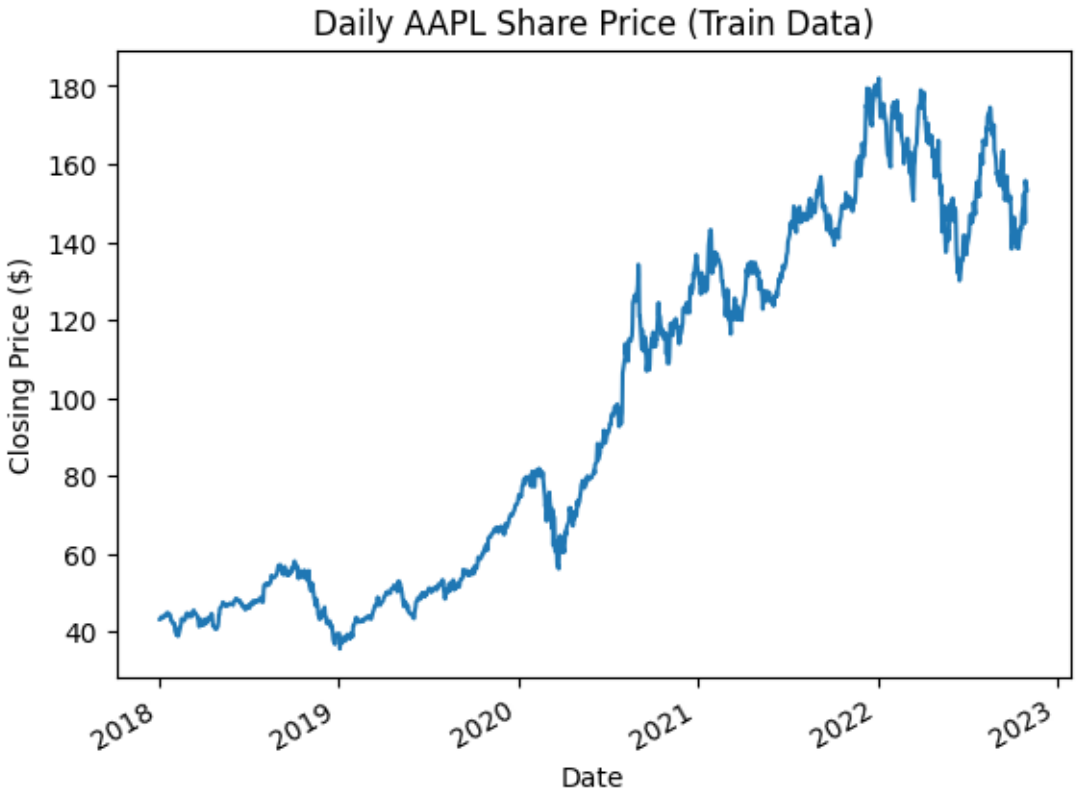
[50]:
```
all_data["Dates"] = pd.to_datetime(all_data["Dates"])
all_data = all_data.sort_values("Dates").reset_index(drop=True)

com_train = all_data[(all_data["Dates"] >= "2018-01-01") & (all_data["Dates"]␣
 ↪<= "2022-10-31")]
com_val   = all_data[(all_data["Dates"] >= "2022-11-01") & (all_data["Dates"]␣
 ↪<= "2023-10-31")]
com_test  = all_data[(all_data["Dates"] >= "2023-11-01") & (all_data["Dates"]␣
 ↪<= "2024-10-31")]
```

Now, we have three seperate sets of data. To prevent data leakage, all data processing will be done to each set seperately. Next, we will perform EDA on only the training data.

First, we build a plot using only the time series stock price data.

3.6 EDA Combined Dataset (Training Data)

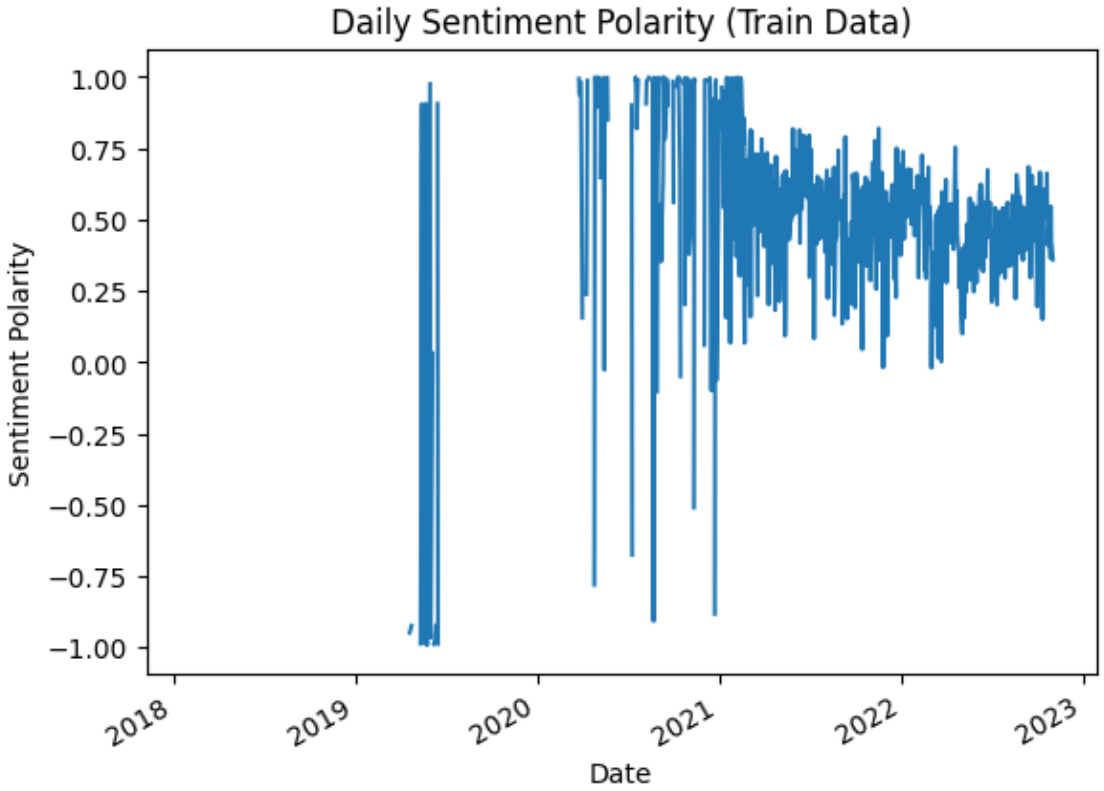
[51]:
```
plt.plot(com_train['Dates'], com_train['AAPL Share Price'])
plt.xlabel('Date')
plt.ylabel('Closing Price ($)')
plt.title('Daily AAPL Share Price (Train Data)')
plt.gcf().autofmt_xdate()
plt.show()
```



During the period of time covered by the train data, AAPL stock fluctuates, but increases overall from July, 2020 to about January 2022. Then, it continues to flucctuate but trends down until July, 2022.

We can also build a plot of the sentiment polarity over the same period of time and compare the two charts.

[52]:
```
plt.plot(com_train['Dates'], com_train['sentiment_polarity'])
plt.xlabel('Date')
plt.ylabel('Sentiment Polarity')
plt.title('Daily Sentiment Polarity (Train Data)')
plt.gcf().autofmt_xdate()
plt.show()
```



The sentiment polarity plot shows many missing values at the start of the time series. Then, there are some extreme highs and lows. Many of these are close to 1 (maximum) and -1 (minimum) and a few are exaclty 1.

There are some low outliers in 2020. These could be explained by the COVID-19 pandemic, which brought uncertainty to the stock market and the economy as a whole.

Most of the sentiment scores, especially in the latter half, are overall positive.

Also, since we know that sentiment polarity scores in this dataset are all in the -1 to 1 range, we can confirm that there are no illogical values in the train data by looking at this plot.

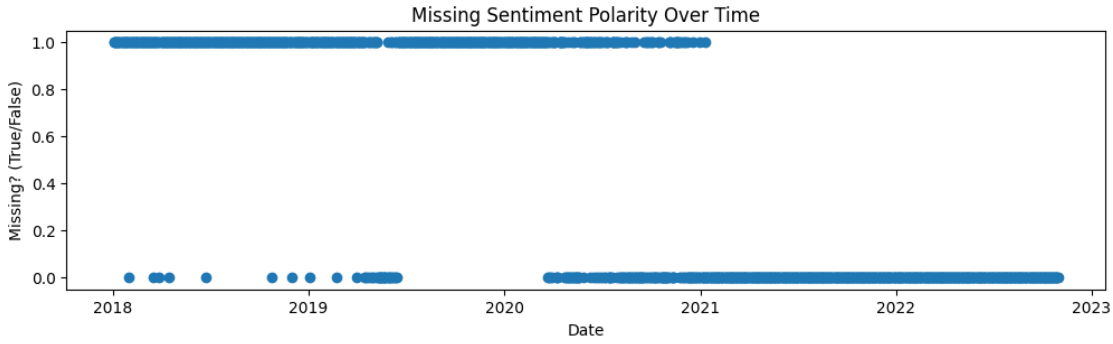
Next, we can examine the extent of missing values, specifically within the sentiment polarity feature.

[53]:
```
com_train['sentiment_polarity'].isna().sum()
```

[53]:
```
np.int64(608)
```

Overall, there are 608 missing values for sentiment polarity within the train data and no missing values for AAPL stock price. Next, we can visualize where exactly these missing values occur in the time series data.

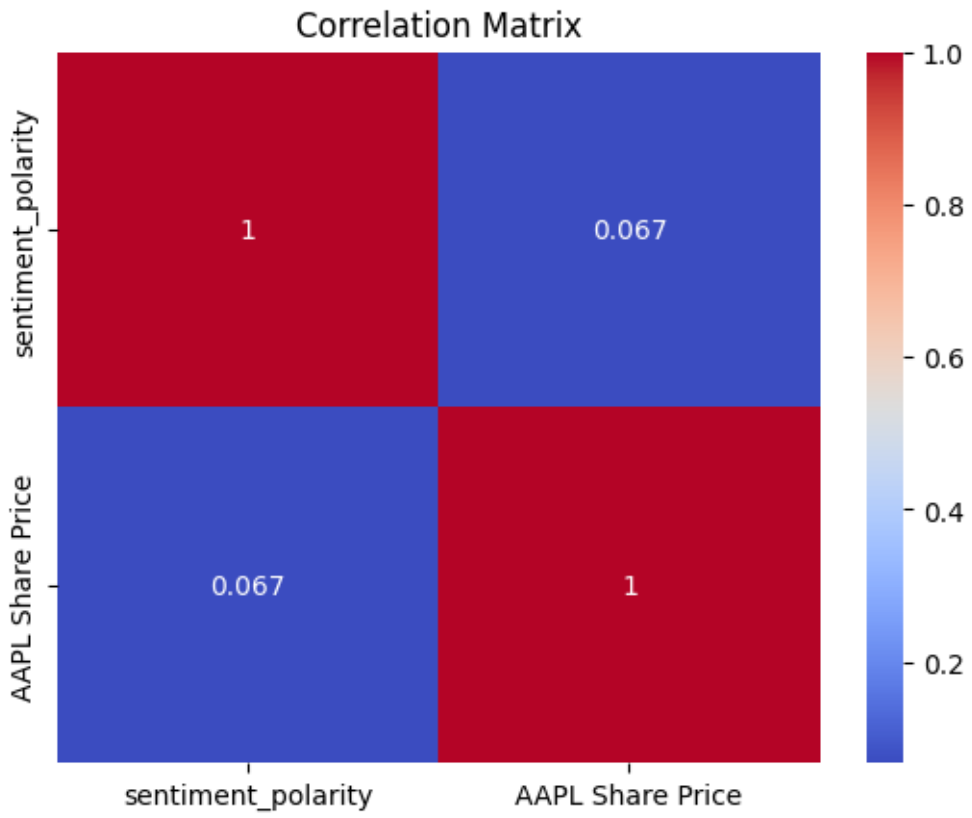
[54]:
```
plt.figure(figsize=(12,3))
plt.plot(
    com_train['Dates'],
    com_train['sentiment_polarity'].isna(),
    marker='o',
    linestyle='None'
)
plt.title("Missing Sentiment Polarity Over Time")
plt.xlabel("Date")
plt.ylabel("Missing? (True/False)")
plt.show()
```



As expected form the previous plot, most of the 608 missing values occur in the earlier months of the train data.

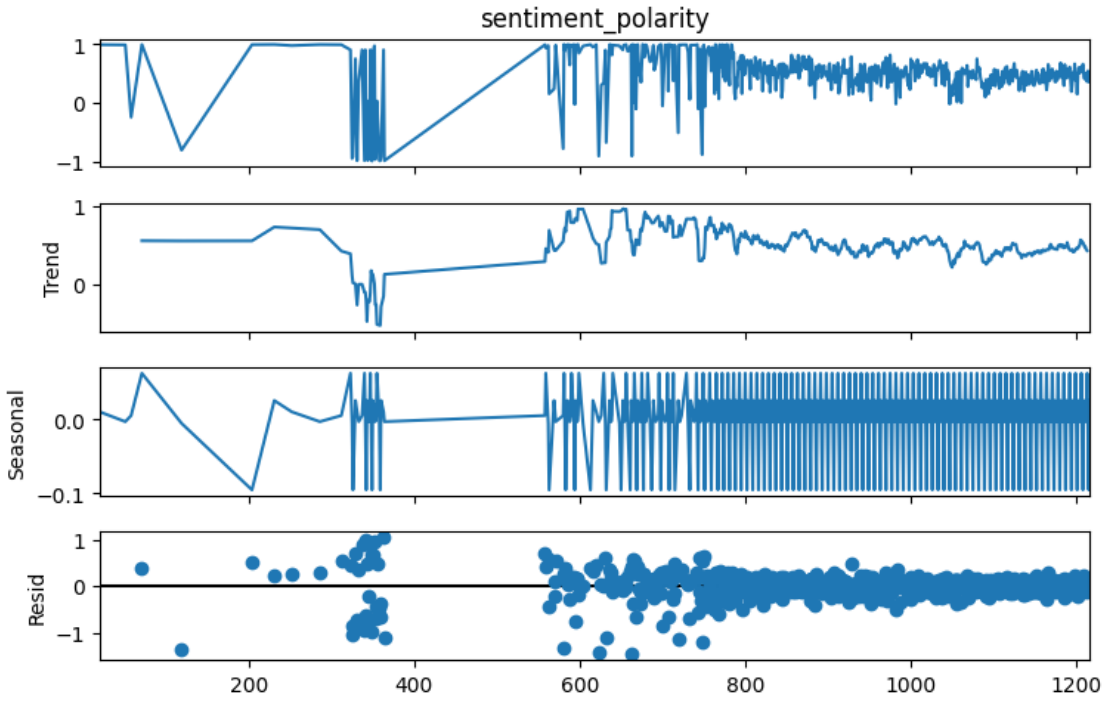
We can also view a correlation matrix to look at how our two variables are correlated.

[55]:
```
corr_matrix = com_train[["sentiment_polarity", "AAPL Share Price"]].corr()

sns.heatmap(corr_matrix, annot=True, cmap="coolwarm")
plt.title("Correlation Matrix")
plt.show()
```



Finally, we can observe a seasonal compostion of the train data.

[56]:
```
y = com_train['sentiment_polarity'].dropna()

decomp = seasonal_decompose(y, model='additive', period=7)
decomp.plot()
plt.gcf().set_size_inches(8, 5)
plt.show()
```



4 Week 4: Make Data Model Ready

4.1 Sentiment Dataset Cleaning

In order to make the sentiment dataset ready for modeling we will first combine our title and content columns into one full text column. We will remove the columns (title, content, sentiment_polarity, net sentiment, title_length, title_word_count, content_length, content_word_count) that will not be necessary for training our model and aggregate ful_text column by date to create a daily text feature for modeling and aggregate sentiment _neg, sentiment_neu and sentiment_pos by mean for each date. This will create a daily summary of the news articles and their sentiment scores.

Combine each dataset (train, validation and test) with the corresponding dates in timeseries dataset (ts_train, ts_val, ts_test) to create the final training datasets that contains the daily aggregated full_text, sentiment_neg, sentiment_neu, and sentiment_pos along with the AAPL Share Price for each date. We use time series dates as the main column dates so that all sentiment datasets dates are aligned with ts_train dates. This will allow us to have a complete dataset for modeling that includes both the text features and the target variable.

Finally we will fill missing data in the full_text column by replacing NaN values with a blank string to avoid putting in placeholders that would otherwise introduce noice into the model. We will forward fill sentiment values up to 7 days as news diminishes overtime and will leave the remaining values at 0.

[57]:
```
#Combine title and content into a single text column for NLP processing and␣
 ↪consistency across train, val, and test sets.
raw_train['full_text'] = raw_train['title'] + ' ' + raw_train['content']
raw_test['full_text'] = raw_test['title'] + ' ' + raw_test['content']
raw_val['full_text'] = raw_val['title'] + ' ' + raw_val['content']
```

[58]:
```
# Remove unnecessary columns from train, val, and test sets to focus on text␣
 ↪and sentiment features.
Raw_X_train = raw_train.drop(columns=['title', 'content', 'sentiment_polarity',␣
 ↪'Net Sentiment', 'title_length', 'title_word_count', 'content_length',␣
 ↪'content_word_count'])
Raw_X_val = raw_val.drop(columns=['title', 'content', 'sentiment_polarity',␣
 ↪'Net Sentiment'])
Raw_X_test = raw_test.drop(columns=['title', 'content', 'sentiment_polarity',␣
 ↪'Net Sentiment'])
```

[59]:
```
# print out the columns and data types for each dataset to ensure they are␣
 ↪consistent and correctly formatted for modeling.
print('Training set columns and data types:')
for col in Raw_X_train.columns:
    print(f'Column: {col} , Data Type: {Raw_X_train[col].dtype}')

print('\nValidation set columns and data types:')
for col in Raw_X_val.columns:
    print(f'Column: {col} , Data Type: {Raw_X_val[col].dtype}')

print('\nTest set columns and data types:')
for col in Raw_X_test.columns:
    print(f'Column: {col} , Data Type: {Raw_X_test[col].dtype}')
```

```
Training set columns and data types:
Column: date , Data Type: datetime64[ns]
Column: symbols , Data Type: object
Column: sentiment_neg , Data Type: float64
Column: sentiment_neu , Data Type: float64
Column: sentiment_pos , Data Type: float64
Column: full_text , Data Type: object

Validation set columns and data types:
Column: date , Data Type: datetime64[ns]
Column: symbols , Data Type: object
Column: sentiment_neg , Data Type: float64
Column: sentiment_neu , Data Type: float64
Column: sentiment_pos , Data Type: float64
Column: full_text , Data Type: object

Test set columns and data types:
```

```
Column: date , Data Type: datetime64[ns]
Column: symbols , Data Type: object
Column: sentiment_neg , Data Type: float64
Column: sentiment_neu , Data Type: float64
Column: sentiment_pos , Data Type: float64
Column: full_text , Data Type: object
```

[60]:
```python
# Aggregate columns by date to create a daily text feature for modeling
# Aggregate sentiment columns by mean for each date.
Raw_X_train_agg = Raw_X_train.groupby('date').agg({
    'full_text': lambda x: ' '.join(x),  # Combine all text for the day
    'sentiment_neg': 'mean',
    'sentiment_neu': 'mean',
    'sentiment_pos': 'mean'
})

Raw_X_val_agg = Raw_X_val.groupby('date').agg({
    'full_text': lambda x: ' '.join(x),  # Combine all text for the day
    'sentiment_neg': 'mean',
    'sentiment_neu': 'mean',
    'sentiment_pos': 'mean'
})

Raw_X_test_agg = Raw_X_test.groupby('date').agg({
    'full_text': lambda x: ' '.join(x),  # Combine all text for the day
    'sentiment_neg': 'mean',
    'sentiment_neu': 'mean',
    'sentiment_pos': 'mean'
})
```

[61]:
```python
Raw_X_train_agg.head(10)
```

[61]:

date	full_text	sentiment_neg	sentiment_neu	sentiment_pos
2018-01-31	Investor Expectations to Drive Momentum within…	0.009		
2018-03-16	Top 100 Reputable Companies Around the Globe A…	0.011		
2018-03-27	Universal Display Corporation Stock Is Way Und…	0.134		
2018-04-16	Detailed Research: Economic Perspectives on Ge…	0.008		
2018-06-22	Apple iPhone Spared Tariffs, But Could Face Ch…	0.138		
2018-10-23	Report: Exploring Fundamental Drivers Behind T…	0.008		
2018-11-30	New Research Coverage Highlights Omeros, Apple…	0.009		
2019-01-03	Apple, AAPL Investment Losses Alert: Bernstein…	0.044		
2019-02-22	Factors of Influence in 2019, Key Indicators a…	0.009		
2019-04-01	Report: Developing Opportunities within Apple,…	0.008		

```
2018-01-31          0.9370          0.054
2018-03-16          0.9170          0.072
2018-03-27          0.7640          0.101
2018-04-16          0.9300          0.062
2018-06-22          0.8620          0.000
2018-10-23          0.9410          0.051
2018-11-30          0.9250          0.066
2019-01-03          0.8275          0.129
2019-02-22          0.9280          0.064
2019-04-01          0.9430          0.049
```

[62]:
```
# Combine each dataset with corresponding stock price data.
Final_Raw_X_train = pd.merge(ts_train[['Dates', 'AAPL Share Price']],␣
 ↪Raw_X_train_agg, left_on='Dates', right_index=True, how='left')
Final_Raw_X_val = pd.merge(ts_val[['Dates', 'AAPL Share Price']],␣
 ↪Raw_X_val_agg, left_on='Dates', right_index=True, how='left')
Final_Raw_X_test = pd.merge(ts_test[['Dates', 'AAPL Share Price']],␣
 ↪Raw_X_test_agg, left_on='Dates', right_index=True, how='left')
```

[63]:
```
#For Nan values in the full_text column, fill with an empty string to allow for␣
 ↪NLP processing without errors.
Final_Raw_X_train['full_text'] = Final_Raw_X_train['full_text'].fillna('')
Final_Raw_X_val['full_text'] = Final_Raw_X_val['full_text'].fillna('')
Final_Raw_X_test['full_text'] = Final_Raw_X_test['full_text'].fillna('')

# For Nan values in the sentiment columns, forwardfill up to 7 days ahead and␣
 ↪replace remaining NaNs with 0.
for col in ['sentiment_neg', 'sentiment_neu', 'sentiment_pos']:
    Final_Raw_X_train[col] = Final_Raw_X_train[col].fillna(method='ffill',␣
 ↪limit=7).fillna(0)
    Final_Raw_X_val[col] = Final_Raw_X_val[col].fillna(method='ffill', limit=7).
 ↪fillna(0)
    Final_Raw_X_test[col] = Final_Raw_X_test[col].fillna(method='ffill',␣
 ↪limit=7).fillna(0)
```

```
/tmp/ipykernel_2500111/1825454038.py:8: FutureWarning: Series.fillna with
'method' is deprecated and will raise in a future version. Use obj.ffill() or
obj.bfill() instead.
  Final_Raw_X_train[col] = Final_Raw_X_train[col].fillna(method='ffill',
limit=7).fillna(0)
/tmp/ipykernel_2500111/1825454038.py:9: FutureWarning: Series.fillna with
'method' is deprecated and will raise in a future version. Use obj.ffill() or
obj.bfill() instead.
  Final_Raw_X_val[col] = Final_Raw_X_val[col].fillna(method='ffill',
limit=7).fillna(0)
/tmp/ipykernel_2500111/1825454038.py:10: FutureWarning: Series.fillna with
'method' is deprecated and will raise in a future version. Use obj.ffill() or
```

```
obj.bfill() instead.
  Final_Raw_X_test[col] = Final_Raw_X_test[col].fillna(method='ffill',
limit=7).fillna(0)
```

[64]:
```
# Create stock price direction target variable: 'up' if current day's price is␣
↪higher than previous day, 'down' if lower or equal.
Final_Raw_X_train['Stock Price Direction'] = (Final_Raw_X_train['AAPL Share␣
↪Price'] > Final_Raw_X_train['AAPL Share Price'].shift(1)).map({True: 'up',␣
↪False: 'down'})
Final_Raw_X_val['Stock Price Direction'] = (Final_Raw_X_val['AAPL Share Price']␣
↪> Final_Raw_X_val['AAPL Share Price'].shift(1)).map({True: 'up', False:␣
↪'down'})
Final_Raw_X_test['Stock Price Direction'] = (Final_Raw_X_test['AAPL Share␣
↪Price'] > Final_Raw_X_test['AAPL Share Price'].shift(1)).map({True: 'up',␣
↪False: 'down'})
```

[65]:
```
Final_Raw_X_train.head(10)
```

[65]:

	Dates	AAPL Share Price	full_text	sentiment_neg	sentiment_neu	sentiment_pos	Stock Price Direction
0	2018-01-02	43.07		0.0	0.0	0.0	down
1	2018-01-03	43.06		0.0	0.0	0.0	down
2	2018-01-04	43.26		0.0	0.0	0.0	up
3	2018-01-05	43.75		0.0	0.0	0.0	up
4	2018-01-08	43.59		0.0	0.0	0.0	down
5	2018-01-09	43.58		0.0	0.0	0.0	down
6	2018-01-10	43.57		0.0	0.0	0.0	down
7	2018-01-11	43.82		0.0	0.0	0.0	up
8	2018-01-12	44.27		0.0	0.0	0.0	up
9	2018-01-16	44.05		0.0	0.0	0.0	down

[66]:
```
Final_Raw_X_train.tail(10)
```

[66]:

	Dates	AAPL Share Price
1207	2022-10-18	143.75
1208	2022-10-19	143.86

```
1209 2022-10-20            143.39
1210 2022-10-21            147.27
1211 2022-10-24            149.45
1212 2022-10-25            152.34
1213 2022-10-26            149.35
1214 2022-10-27            144.80
1215 2022-10-28            155.74
1216 2022-10-31            153.34

                                                full_text  sentiment_neg  \
1207  10 Best S&amp;amp;P 500 Stocks to Buy According to…       0.023971
1208  Apple (AAPL) Expands Portfolio With Redesigned…       0.046526
1209  Snap Stock Tumbles as Sales Growth Slows Furth…       0.053167
1210  Why Snap Stock Got Crushed Early Friday The ca…       0.036026
1211  Apple raises prices of its services as media e…       0.036469
1212  Investors watch for recession clues in Big Tec…       0.038718
1213  SONY Gears Up to Report Q2 Earnings: Here's Wh…       0.044714
1214  Tech Live: Inside Appleâ s Thinking on Privac…       0.037659
1215  US STOCKS-Wall Street jumps about 2% on upbeat…       0.050182
1216  Chinese workers flee COVID lockdown at huge iP…       0.045520

      sentiment_neu  sentiment_pos Stock Price Direction
1207       0.883971       0.092000                    up
1208       0.850947       0.102632                    up
1209       0.834833       0.111667                  down
1210       0.865795       0.098205                    up
1211       0.882906       0.080500                    up
1212       0.873256       0.087923                    up
1213       0.863750       0.091536                  down
1214       0.869439       0.092902                  down
1215       0.854227       0.095621                    up
1216       0.870900       0.083720                  down
```

4.2 Price Dataset Cleaning

Next, we will clean the price dataset. The price dataset only contains time series AAPL price. The main purpose of this dataset will be to predict the direction of AAPL stock using only historical price data.

Our EDA indicated that there were no noticeable outliers in the stock price data. As a result, no measures were taken to remove outliers.

There are no missing values in the training, validation, or test data, so the step of imputing missing values can be skipped altogether.

Finally, the dataset is transformed to prepare it for a classification problem. With this dataset, we will use the directional movement of AAPL stock in previous days to predict future directional movement.

[67]:
```python
def price_cleaning(df_price: pd.DataFrame) -> pd.DataFrame:
    df_price = df_price.copy()
    df_price = df_price.sort_values("Dates")

    # Create price lags
    for i in range(1, 6):
        df_price.loc[:, f"aapl_price_lag{i}"] = df_price["AAPL Share Price"].
 ↪shift(i)

    # Create direction lags (1 = Up, 0 = Down)
    for i in range(1, 5):
        df_price.loc[:, f"aapl_price_direction_lag{i}"] = (
            (df_price[f"aapl_price_lag{i}"] - df_price[f"aapl_price_lag{i+1}"])␣
 ↪> 0
        ).astype(int)

    # Create target direction variable
    df_price.loc[:, "Stock Price Direction"] = (
        (df_price["AAPL Share Price"] - df_price["aapl_price_lag1"]) > 0
    ).map({True: "Up", False: "Down"})

    # Finalize columns
    final_cols = [
        "Stock Price Direction", "Dates",
        "aapl_price_direction_lag1", "aapl_price_direction_lag2",
        "aapl_price_direction_lag3", "aapl_price_direction_lag4"
    ]
    return df_price.loc[:, final_cols].reset_index(drop=True)
```

The training, validation, and test sets are cleaned seperately, but in the same fashion, to prevent data leakage.

[68]:
```python
# Apply to training set
cleaned_ts_train = price_cleaning(ts_train)

# Apply to validation and test sets seperately to aviod data leakage
cleaned_ts_val = price_cleaning(ts_val)
cleaned_ts_test = price_cleaning(ts_test)
cleaned_ts_train.head()
```

[68]:

	Stock Price Direction	Dates	aapl_price_direction_lag1	\
0	Down	2018-01-02	0	
1	Down	2018-01-03	0	
2	Up	2018-01-04	0	
3	Up	2018-01-05	1	
4	Down	2018-01-08	1	

```
   aapl_price_direction_lag2  aapl_price_direction_lag3  \
0                          0                          0
1                          0                          0
2                          0                          0
3                          0                          0
4                          1                          0

   aapl_price_direction_lag4
0                          0
1                          0
2                          0
3                          0
4                          0
```

4.3 Combined Dataset Cleaning

This section of the notebook will ensure that the combined dataset (including both the text and price series data) is cleaned and model ready. We encapsulate all cleaning and preprocessing methods in one data_cleaning() function, which will be called identically on the validation and test datasets.

The first decision is whether to remove outliers. Referencing the line graph above, there are a few outlying observations with a negative sentiment polarity that are extremely different from the preceding and following days. However, there is still some value in keeping these observations. Many of them coincide with dips in the AAPL stock price, and this could be very value information for the classification model. As a result, all outliers will remain in this dataset.

The next decision is on missing data. All missing data comes from the sentiment news dataset as there are some days where no articles on the AAPL stock were published. One basic assumption we can make is that the sentiment in media will be relatively stable until a new article is published with new sentiment. To follow this logic, we institute a forward fill function where the most recent sentiment score will fill in any missing observation. However, we put a cap of a week, insinuating that articles more than a week old are too antiquated to reflect current sentiment.

The next decision is whether to standardize the sentiment news scores. These scores are already bounded (between -1 and 1) and centered (0 is a perfectly neutral post). The current distribution is also very intuitive, a negative number means a negative sentiment, a positive number is a positive sentiment. As a result, this variable does not need to be standardized further, it can remain as is. This may be reconsidered if we use a neural network model as these models strongly prefer standardized data.

[69]:
```
def data_cleaning(df):
    #forward fill, but only up to a week
    df['sentiment_polarity'] = df['sentiment_polarity'].ffill(limit=7)

    #create lags for price and sentiment polarity variables. Recreate␣
 ↪methodology from price datset
    for i in range(1, 6):
        #create lags for both price and sentiment polarity variables
```

```
        df[f'aapl_price_lag{i}'] = df['AAPL Share Price'].shift(i)
        df[f'sentiment_polarity_lag{i}'] = df['sentiment_polarity'].shift(i)

        #create a price direction lag variable. These are "one-hot encoded"␣
↪since they are binary. 1 = UP, 0 = DOWN
        if i > 1:
            df[f'aapl_price_direction_lag{i-1}'] =  (df[f'aapl_price_lag{i-1}']␣
↪- df[f'aapl_price_lag{i}'] > 0).astype(int)

    #create a price direction variable, this is the target variable as␣
↪discussed in Week 3
    df['Stock Price Direction'] = (df['AAPL Share Price'] -␣
↪df['aapl_price_lag1']).apply(lambda x: 'Up' if x > 0 else 'Down')

    #drop rows that are missing lagged variables
    df.dropna(inplace = True)

    #drop duplicates, but there shouldn't be any because of the primary key date
    df.drop_duplicates(inplace = True)

    #pick the final columns for our dataset
    final_df = df[['Stock Price Direction', 'Dates', 'sentiment_polarity_lag1',␣
↪'sentiment_polarity_lag2', 'sentiment_polarity_lag3',␣
↪'sentiment_polarity_lag4', 'sentiment_polarity_lag5',␣
↪'aapl_price_direction_lag1', 'aapl_price_direction_lag2',␣
↪'aapl_price_direction_lag3', 'aapl_price_direction_lag4']]

    return final_df
```

[70]:
```
#perform data cleaning for training dataset
cleaned_com_train = data_cleaning(com_train)
cleaned_com_train.head(10)
```

[70]:

	Stock Price Direction	Dates	sentiment_polarity_lag1	\
25	Down	2018-02-07	0.995	
26	Down	2018-02-08	0.995	
27	Up	2018-02-09	0.995	
56	Down	2018-03-23	0.991	
57	Up	2018-03-26	0.991	
58	Down	2018-03-27	0.991	
59	Down	2018-03-28	-0.244	
60	Up	2018-03-29	-0.244	
61	Down	2018-04-02	-0.244	
62	Up	2018-04-03	-0.244	

```
    sentiment_polarity_lag2  sentiment_polarity_lag3  sentiment_polarity_lag4  \
25                    0.995                    0.995                    0.995
26                    0.995                    0.995                    0.995
27                    0.995                    0.995                    0.995
56                    0.991                    0.991                    0.991
57                    0.991                    0.991                    0.991
58                    0.991                    0.991                    0.991
59                    0.991                    0.991                    0.991
60                   -0.244                    0.991                    0.991
61                   -0.244                   -0.244                    0.991
62                   -0.244                   -0.244                   -0.244

    sentiment_polarity_lag5  aapl_price_direction_lag1  \
25                    0.995                          1
26                    0.995                          0
27                    0.995                          0
56                    0.991                          0
57                    0.991                          0
58                    0.991                          1
59                    0.991                          0
60                    0.991                          0
61                    0.991                          1
62                    0.991                          0

    aapl_price_direction_lag2  aapl_price_direction_lag3  \
25                          0                          0
26                          1                          0
27                          0                          1
56                          0                          0
57                          0                          0
58                          0                          0
59                          1                          0
60                          0                          1
61                          0                          0
62                          1                          0

    aapl_price_direction_lag4
25                          1
26                          0
27                          0
56                          0
57                          0
58                          0
59                          0
60                          0
61                          1
62                          0
```

[71]:
```
#perform data cleaning for validation set
cleaned_com_val = data_cleaning(com_val)
cleaned_com_val.head(10)
```

[71]:

	Stock Price Direction	Dates	sentiment_polarity_lag1	sentiment_polarity_lag2	sentiment_polarity_lag3	sentiment_polarity_lag4	sentiment_polarity_lag5	aapl_price_direction_lag1	aapl_price_direction_lag2
1222	Up	2022-11-08	0.102262	0.444375	0.347913	0.283481	0.326217	1	0
1223	Down	2022-11-09	0.398720	0.102262	0.444375	0.347913	0.283481	1	1
1224	Up	2022-11-10	0.399429	0.398720	0.102262	0.444375	0.347913	0	1
1225	Up	2022-11-11	0.482846	0.399429	0.398720	0.102262	0.444375	1	0
1226	Down	2022-11-14	0.707300	0.482846	0.399429	0.398720	0.102262	1	1
1227	Up	2022-11-15	0.284654	0.707300	0.482846	0.399429	0.398720	0	1
1228	Down	2022-11-16	0.404769	0.284654	0.707300	0.482846	0.399429		
1229	Up	2022-11-17	0.624091	0.404769	0.284654	0.707300	0.482846		
1230	Up	2022-11-18	0.613143	0.624091	0.404769	0.284654	0.707300		
1231	Down	2022-11-21	0.465308	0.613143	0.624091	0.404769	0.284654		

```
1228                          1                          0
1229                          0                          1
1230                          1                          0
1231                          1                          1

      aapl_price_direction_lag3  aapl_price_direction_lag4
1222                          0                          0
1223                          0                          0
1224                          1                          0
1225                          1                          1
1226                          0                          1
1227                          1                          0
1228                          1                          1
1229                          0                          1
1230                          1                          0
1231                          0                          1
```

[72]:
```
#perform data cleaning for test dataset
cleaned_com_test = data_cleaning(com_test)
cleaned_com_test.head(10)
```

[72]:
```
     Stock Price Direction      Dates  sentiment_polarity_lag1  \
1473                    Up 2023-11-08                 0.451958
1474                  Down 2023-11-09                 0.413867
1475                    Up 2023-11-10                 0.481171
1476                  Down 2023-11-13                 0.556571
1477                    Up 2023-11-14                 0.572346
1478                    Up 2023-11-15                 0.652667
1479                    Up 2023-11-16                 0.757655
1480                  Down 2023-11-17                 0.732615
1481                    Up 2023-11-20                 0.557379
1482                  Down 2023-11-21                 0.130737

      sentiment_polarity_lag2  sentiment_polarity_lag3  \
1473                 0.493862                 0.410016
1474                 0.451958                 0.493862
1475                 0.413867                 0.451958
1476                 0.481171                 0.413867
1477                 0.556571                 0.481171
1478                 0.572346                 0.556571
1479                 0.652667                 0.572346
1480                 0.757655                 0.652667
1481                 0.732615                 0.757655
1482                 0.557379                 0.732615

      sentiment_polarity_lag4  sentiment_polarity_lag5  \
1473                 0.458235                 0.558027
```

```
1474                0.410016                0.458235
1475                0.493862                0.410016
1476                0.451958                0.493862
1477                0.413867                0.451958
1478                0.481171                0.413867
1479                0.556571                0.481171
1480                0.572346                0.556571
1481                0.652667                0.572346
1482                0.757655                0.652667

      aapl_price_direction_lag1  aapl_price_direction_lag2  \
1473                          1                          1
1474                          1                          1
1475                          0                          1
1476                          1                          0
1477                          0                          1
1478                          1                          0
1479                          1                          1
1480                          1                          1
1481                          0                          1
1482                          1                          0

      aapl_price_direction_lag3  aapl_price_direction_lag4
1473                          0                          1
1474                          1                          0
1475                          1                          1
1476                          1                          1
1477                          0                          1
1478                          1                          0
1479                          0                          1
1480                          1                          0
1481                          1                          1
1482                          1                          1
```